

Git style Branching and Merging for AI Coding Chat

Description

Large language models are increasingly used for programming support, but coding performance often decreases and conversation gets difficult to manage for both the LLM and the programmer once the conversation grows longer or more complex in intent. In practice, users may want to try several implementation directions, revisit earlier assumptions, compare alternative fixes, or preserve promising partial ideas without losing the rest of the conversation. Standard linear chat interfaces do not support this well. This project explores whether a version control inspired interaction model, centered on branching and merging, can better support non linear coding workflows.

The project will investigate **how branching and merging (per which granularity) should be designed** in AI coding chat. One possible view is to divide branches into alternative conversation paths. Another is to divide them as engineering artifacts, such as plans, code candidates, debugging strategies, or review decisions. Similarly, merging may mean combining textual context, selected code fragments, summaries of prior attempts, or user chosen outputs from multiple threads. The project does not need to be restricted to one interpretation at the start. Part of the work is to identify which **granularity** makes the most sense for real coding sessions.

A prototype system is expected, but the exact feature scope should remain flexible. A lightweight version may support creating branches from earlier messages and navigating across threads. A more advanced version may include visual branch structure, branch comparison, selective reintegration, or merge support. The final design can be adjusted according to time, technical feasibility, and the student's interests. The central goal is not to build as many features as possible in the system, but to study whether **non-linear conversation** structure helps users manage complexity in programming tasks.

The project may be evaluated in different ways. One direction is a usability oriented study focusing on learnability, perceived control, and user satisfaction. Another can be a task-based study focusing on coding efficiency, solution quality, number of retries, or how well users can manage alternatives and recover from unproductive paths. Mixing evaluation is also possible.

The expected contribution is a better understanding of how branching and merging can be adapted from **code versioning** to **Human-AI conversation support**. This may result in a prototype, empirical findings on how programmers use structured chat, and design implications for future AI coding interfaces. The project is suitable for students interested in HCI, developer tools, AI assisted programming, or interactive systems.

Related Works

<https://arxiv.org/abs/2512.22790>

<https://arxiv.org/abs/2512.13914>
[SEMINAR] <https://arxiv.org/abs/2505.06120>
<https://github.com/ericmjl/canvas-chat>

Research goal

The goal of this project is to design and implement an in-IDE AI coding assistant chat prototype that supports branching and merging for both message and artifact, and to examine whether this non linear interaction model helps programmers manage complex coding workflows better than a standard linear chat interface. The focus is not on adding many interfaces and/or features, but on understanding whether structured branching and selective merging help users explore alternatives, recover from unproductive paths, compare implementation directions, and reuse useful intermediate results during programming tasks.

Research questions

RQ1. How should branching and merging be represented in an AI coding chat for programmers working in an IDE?

This question studies the right unit of branching and reintegration. Message and artifact should not be treated as conflicting units. Instead, each message is tied to a concrete engineering state or artifact change, and branch history should preserve that relation. The design question is therefore how to model artifact evolution in a way that remains understandable and manageable in chat messages for programmers.

RQ2. Compared with a standard linear AI coding chat, does a branching and merging enabled interface improve task performance in complex programming tasks?

This question targets the outcomes of the chat interface. Relevant outcomes include task completion, correctness, solution quality, time to reach a working solution, and the ability to recover from failed attempts.

RQ3. How do programmers actually use branching and merging during coding, and what usage patterns are associated with successful problem solving?

This question targets the process behavior. It examines when users create branches, which branches are revisited, what gets merged back, how often users abandon a path, and whether branching is used mainly for debugging, comparing designs, or protecting partial progress.

Outline of the solution

The prototype should be an IDE (TBD VSCode or JetBrains) integrated AI-chat system with a feature set. A readily-existing code completion plugin can be used (e.g., <https://github.com/AISE-TUdelft/code4me2>).

At minimum, it should allow the user to branch from an earlier message or state, navigate across branches, inspect the relation between a branch and the current main line, and selectively merge results back. The key design point is that a branch should preserve both the conversational context and its associated code state.

For merging, the system should support selective code merge rather than full thread concatenation. In practice, this means that a user should be able to merge back a chosen result, such as a code change, a patch, a refined plan, or a resolved debug, without collapsing the entire conversation history into one flat thread. The reason for the action should be visible, so the user can tell where a merged result came from.

A sensible minimal feature scope would be:

branch from a prior point, switch between branches, view branch lineage, inspect branch specific code state, and merge selected outputs back into the active path.

A reasonable stretch scope would be:

branch comparison, lightweight summaries, merge previews, or conflict indicators when two branches modify the same code region.

How to evaluate it

Participants with programming experience complete comparable coding tasks using two conditions: a standard linear AI chat interface and the branching and merging prototype. This design keeps the comparison focused and reduces individual skill differences.

The tasks should be chosen so that non linear exploration is naturally useful. Good examples are debugging tasks with multiple plausible causes, feature implementation tasks with more than one reasonable design path, or repair tasks where an early attempt may fail and users need to recover without losing useful partial work.

Task performance can be measured through completion rate, correctness, test pass rate, solution quality, total task time, and/or time to first working version. Process behavior can be measured through branch creation frequency, number of revisited branches, number of abandoned paths, number of merged outputs, time spent on each branch, and the kinds of artifacts users choose to merge back, etc. Screen recordings, event logs, and repository or file diffs can support this analysis.

A short post task questionnaire and interview could be included. The questionnaire can ask about perceived control and cognitive burden (NASA-TLX). The interview can also focus on concrete use moments: when branching felt useful, when it felt unnecessary, and whether merging helped preserve progress or introduced confusion.

Timeline

W2/3

- **Work**
 - Finalize the project scope, research focus, and first branching/merging model specs.
 - Define the MVP feature set, keeping the scope realistic for the available time.
 - Select evaluation tasks, datasets, and the study direction.
 - Prepare initial workflow scenarios and low fidelity interface sketches.
 - Build the prototype foundation, including branching from earlier messages, branch navigation, and basic branch state storage.
 -
- **Deliverables**
 - Final scope statement and MVP feature list.
 - Initial task scenarios and/or selected datasets.
 - Low fidelity mockup or wireframes.
 - Basic system architecture and storage/state handling design.
 - Early working prototype showing one complete branching workflow, plus a list of open technical issues.

W4

- **Work**
 - Complete the MVP workflow
 - Add a simple merge / reintegration mechanism
 - Run internal pilot tests
 - Identify usability problems and workflow friction
 - Refine the formal evaluation setup
- **Deliverables**
 - MVP prototype
 - Pilot test notes
 - Revised design decisions
 - Frozen candidate task set for evaluation

W5

- **Work**
 - Prepare for and conduct midpoint check
 - Present the design rationale and current prototype
 - Report what was learned from pilots
 - Freeze the post midpoint plan and study protocol
- **Deliverables**
 - Midpoint presentation / demo
 - Progress summary
 - Stable evaluation protocol
 - Finalized implementation plan for the remaining weeks

W6

- **Work**
 - Stabilize the prototype
 - Run the main user study / testing

- Collect usage logs, task results, and feedback
- Restrict development to reliability fixes only
- **Deliverables**
 - Study dataset
 - Cleaned logs / observation notes
 - Questionnaire or interview records
 - Stable study version of the prototype

W7

- **Work**
 - Analyze the results
 - Identify whether branching helped exploration, comparison, and recovery
 - Summarize usability and task performance findings
 - Select representative examples for the report
- **Deliverables**
 - Analysis summary
 - Main findings
 - Draft figures / tables
 - Selected qualitative cases

W8

- **Work**
 - Write and finalize the report draft
 - Consolidate design, method, results, and limitations
 - Polish figures and prototype documentation
 - Fix only critical bugs if needed
- **Deliverables**
 - Final draft submission
 - Submission ready figures and tables
 - Stable prototype build or demo video
 - Organized appendix / repository materials

W9

- **Work**
 - Revise based on peer feedback
 - Improve clarity of argument, figures, and contribution
 - Refine presentation structure
 - Prepare final demo flow and backup recording
- **Deliverables**
 - Revised final report
 - Updated presentation slides
 - Final demo script
 - Revision summary

W10

- **Work**
 - Rehearse and give the final presentation
 - Demonstrate the prototype clearly
 - Emphasize problem, design choice, and evaluation results
- **Deliverables**
 - Final presentation slides
 - Final demo
 - Final submitted project package